

Two notions of polynomial time on the word-RAM

Lax51 and the current Lax58 example • 6 September 2026

The definitions differ in both their input-size measure and their required word-width threshold. This note compares Lax51's `RamPolytime` with `PolynomialTime` in Lax58's `RamComplexityExample`, not with the unrestricted two-resource predicate `RamComputableWithin`.

For a list x of natural numbers, use the following quantities:

$$\begin{aligned} n &= \text{number of entries in } x, \\ m &= \text{largest entry in } x \text{ (zero for the empty list),} \\ B &= \text{bitSize}(x) = \text{length of Lax51's binary encoding,} \\ M &= \text{inputMagnitude}(x) = 2n + m + 2. \end{aligned}$$

The binary encoding has one separator per entry, followed by that entry's binary digits. Thus $B \geq n$. For example, $[2^{1000}]$ has $n = 1$ but $B = 1002$.

1. Definitions and units

	Lax51: <code>RamPolytime</code>	Lax58: current <code>PolynomialTime</code> example
Time measure	Binary input size B .	Number of list entries n .
Time allowance	$P(B)$ for a polynomial P .	$A(n + 1)^d$ for constants $A, d \in \mathbb{N}$.
Word requirement	$w \geq Q(B)$, where Q is a polynomial. The bound is in <i>bits</i> .	$CM \leq 2^w$, plus payload and address fit, for a constant C . The bound is in <i>numerical capacity</i> .
Input magnitude M	Not used as the size measure; size is the binary encoding length B .	$M = S + m + 1$: structural node count S , plus largest natural payload m , plus one. Here $S = 2n + 1$, so $M = 2n + m + 2$. This is neither bit length nor memory usage.
Physical input	The native list with its length prepended: $n :: x$.	The certified structural arena: a root word and three words per structural node.
Output compared here	The singleton list $[f(x)]$.	The same singleton list $[f(x)]$, because f returns a natural number.
Uniformity	One program, working at every sufficiently large width.	The same quantifier discipline.

Neither definition imposes an upper limit on the actual word width w . Each guarantees a run at *every* width above its sufficient threshold. The question is how small that threshold must be, not how large the machine is allowed to be.

Lax51 requires the native input and exact output to fit at width $Q(B)$, and then requires a run of at most $P(B)$ instructions for every $w \geq Q(B)$. For example, the sufficient width could be B^2 bits, with numerical capacity 2^{B^2} .

For a list of naturals, Lax58's three width premises are exactly

$$m < 2^w, \quad 6n + 3 \leq 2^w, \quad C(2n + m + 2) \leq 2^w.$$

The middle condition addresses the arena's $3S = 6n + 3$ memory words. For fixed C , all three conditions hold starting at a width of order $\log(n + m + 2)$. For bounded payloads, this is $O(\log n)$ bits.

2. Why input magnitude contains m , not $\log m$

A w -bit word represents integers from 0 through $2^w - 1$. Storing the value m therefore requires $m < 2^w$, or about $\log_2(m + 1)$ bits. The logarithm appears when solving for the *width*; it does not belong on the capacity side of the inequality.

Accordingly, $C(2n + m + 2) \leq 2^w$ already gives logarithmic dependence of the required width on m . Replacing m by $\log m$ on the left would mix the bit length of a value with the numerical range needed to represent it. The separate payload-fit premise would still be necessary. Input magnitude is a convenient combined scale for address range and payload values, not a claim that either the storage or the number of bits is proportional to m .

3. A function separating the definitions

Consider

$$f(x) = 2^{\text{length}(x)}.$$

For Lax51, use the output adapter $x \mapsto [f(x)]$ to match the natural-number output of the Lax58 example. The value 2^n needs $n + 1$ bits, even though its numerical value is exponential in n .

Why Lax51 accommodates it. The physical input starts with n . A fixed RAM program reads this prefix, sets a register to 1, shifts it left by n bits, writes the result, and halts. It need not inspect the remaining input entries. Left shift is an instruction in the underlying RAM model, so the instruction count is constant. Since $n \leq B$, the polynomial sufficient width $Q(B) = B + 2$ accommodates the input entries, the length prefix, and the output. The program works at every larger width as well.

Why the current Lax58 example excludes it. Let x consist of n zeros. Then $m = 0$, $B = n$, and $M = 2n + 2$. For any proposed constant C , choose the least w for which

$$2^w \geq \max\{6n + 3, C(2n + 2)\}.$$

This width satisfies all three Lax58 premises and is $O(\log n)$. Nevertheless, its required one-word output 2^n fits only if $w \geq n + 1$. For sufficiently large n , these conditions are incompatible. Increasing the time coefficient or degree cannot fix an output that does not fit in a word. Returning several words would change the specified output encoding.

For illustration, take $C = 10$ and $n = 1000$: the premises hold at $w = 15$, because $2^{15} = 32768 \geq 20020$. But the output requires 1001 bits. For any other fixed C , a sufficiently large n produces the same obstruction.

4. Consequence and possible alignment

The definitions are not equivalent, even on zero lists where $B = n$. The general Lax58 predicate is flexible enough to use time bound $P(B)$ and capacity bound $2^{Q(B)}$; its capacity premise then becomes $Q(B) \leq w$. The new `BitPolynomialTime` definition makes this alignment while keeping arena input. For natural-number outputs, its equivalence with Lax51 is now proved by verified translations between arena and native length-prefixed input tapes. Each direction chooses its own polynomial bounds; their degrees are not required to agree. Concretely, arena-to-native conversion sends a time bound $T(B)$ to $66T(B) + 6B + 81$, whereas native-to-arena conversion sends it to $18T(B) + 26$. Both send a sufficient-width bound $Q(B)$ to $Q(B) + 7$. Therefore all nonconstant time degrees are preserved, but a constant arena-time bound can become linear because the current adapter performs linear startup work.

Status. Both the separation and the general equivalence with Lax51 are kernel-checked. See `Lax51.RamPolytime` and `Lax58.RamPolynomialComparison` for the source definitions.